

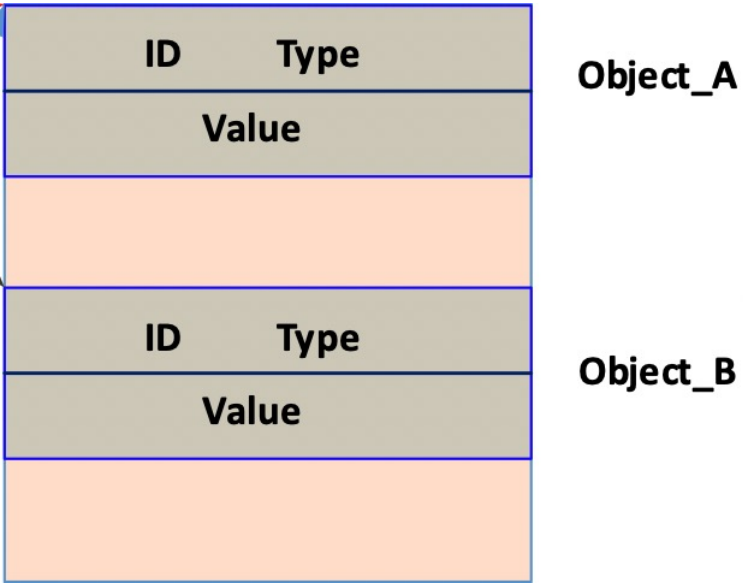
# Python 程序设计

字符串处理

# 复习：Python内置对象类型

| 对象类型       | 示例                                         |
|------------|--------------------------------------------|
| 数字         | 1234, 3.14, <b>3.14e-10</b> , 3+4j         |
| <b>字符串</b> | <b>'swfu', "I'm student", '''Python'''</b> |
| 列表         | [1, 2, 3] ['a','b',[ <b>'c',2</b> ]]       |
| 字典         | {1:'food' ,2:'taste', 3:'import'}          |
| 元组         | (2, -5, 6)                                 |
| 文件         | f=open('data.dat', 'r')                    |
| 集合         | {'a', 'b', 'c'}                            |
| 布尔型        | True, False                                |
| 空类型        | None                                       |
| 编程单元类型     | 函数(def)、模块、类(class)                        |

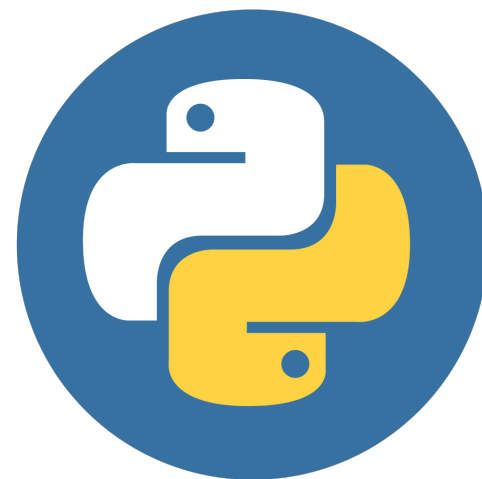
**Literal(字面值)**  
表示某个内置对象类型的固定不变的值，在词法和语法分析时识别其类型



# 为啥要处理字符串

- Word
  - 根据模板生成不同的邮件
- Email
  - 根据不同的类别发送不同的邮件
- Excel
  - 调研问卷的结论处理，生成表格

各种重复劳动



# 定义字符串

"这是一个"

'这是另一个'

"这是A的"

'这是"另一种"的'

""可以换行的 ""也可以

字符串

长成

长这样""

这样""

注意此处的单引号和双引号  
均为英文字符

## 1.4.4 字符串(str)

- 与其他语言一般用双引号来定义字符串字面值不同，Python可以使用**单引号**和**双引号**
  - Python3的字符串默认为Unicode字符串，从而可以包含中文字符串
  - 如果字符串中有双引号，则使用单引号定义，反之亦然

'abc' "Python程序设计基础" "What's your name?" "'Thank you!' she said. ‘

- 单引号**和**双引号**定义的字符串不能跨越多行，字符串中想要有换行需要使用字符转义
- 三引号**'''或'''表示的字符串可以换行，用于长字符串，也用于在程序中表示较长的注释。

```
>>> a = '''first line
line 2
line 3
last line.'''
```

```
>>> a = '''first line
line 2
line 3
last line.'''
>>> a
'first line\nline 2\nline 3\nlast line.'
>>> print(a)
first line
line 2
line 3
last line.
```

- 字符串属于**不可变序列**
- 空串表示为""或'''

# 更复杂的字符串

- 一个字符串中 " 和 ' 都存在
- 或者有回车，Tab

**转义符\**：表示需要后面的字符需要特别对待，不是它本来的意思了

| 转义字符     | 描述                                |
|----------|-----------------------------------|
| \\(在行尾时) | 续行符                               |
| \\       | 反斜杠符号                             |
| \'       | 单引号                               |
| \"       | 双引号                               |
| \a       | 响铃                                |
| \b       | 退格(Backspace)                     |
| \e       | 转义                                |
| \000     | 空                                 |
| \n       | 换行                                |
| \v       | 纵向制表符                             |
| \t       | 横向制表符 ( Tab)                      |
| \r       | 回车                                |
| \f       | 换页                                |
| \oyy     | 八进制数，y 代表 0~7 的字符，例如：\012 代表换行。   |
| \xyy     | 十六进制数，以 \x 开头，yy代表的字符，例如：\x0a代表换行 |
| \other   | 其它的字符以普通格式输出                      |

# 回顾:字符串转义符\

- **转义符\**: 表示需要后面的字符需要特别对待, 不是它本来的意思了

| 特殊字符 | 含义    | 特殊字符 | 含义    |
|------|-------|------|-------|
| \'   | 一个单引号 | \"   | 一个双引号 |
| \\   | 一个\   | \t   | 制表符   |
| \n   | 换行符   | \r   | 回车    |

- 如一个字符串内容有单双引号, 则需要通过在引号前加上转义字符, 表示引号为字符串的一部分, 而不是字符串的结束

```
>>> s = 'he said:"I\'m superman."'
```

```
>>> s
'he said:"I\'m superman."'
>>> print(s)
he said:"I'm superman."
```

```
>>> s2 = "No.\tname\tage\t\n1\tTom\t21"
```

```
>>> s2
'No.\tname\tage\t\n1\tTom\t21'
>>> print(s2)
```

| 转义字符     | 描述                            |
|----------|-------------------------------|
| \\(在行尾时) | 续行符                           |
| \\       | 反斜杠符号                         |
| \'       | 单引号                           |
| \"       | 双引号                           |
| \a       | 响铃                            |
| \b       | 退格(Backspace)                 |
| \e       | 转义                            |
| \000     | 空                             |
| \n       | 换行                            |
| \v       | 纵向制表符                         |
| \t       | 横向制表符                         |
| \r       | 回车                            |
| \f       | 换页                            |
| \yy或\yyy | 八进制数yy代表的字符，例如：'\12' 代表换行     |
| \xyy     | 十六进制数yy代表的字符，例如：'\x0a'代表换行    |
| \other   | 其它的字符以普通格式输出                  |
| \uxxxx   | 描述码点xxxx对应的字符，如：'\u56fd' 是'中' |

```
>>> a = '\t\106\t\x61\t\c'
```

\106 → F

\x61 → a

\c → \c

```
>>> a = '\t\106\t\x61\t\c'
>>> a
"\tF\ta\t\c"
>>> print(a)
'      F      a      \c'
>>> len(a)
8
>>> str1="hello"
>>> len(str1)
5
>>>
```

## Shell实验：

```
>>> s = 'he said: "I\'m superman. "'
```

```
>>> s2 = "No.\tname\tage\t\n1\tTom\t21"
```

```
>>> s3 = '\t\t106\t\tx61\t\tc'
```

```
>>> s4 = u"\U0001F923"
```

```
>>> s4 = u"\U0001F923"
```

```
>>> print(s4)
```



# 原始字符串

- `a="C:\Users\Student1\Desktop\Python"`
- `b=r"C:\Users\Student1\Desktop\Python"`
- `c=r"C:\Users\Student1\Desktop\Python\"`
- 字符串界定符前面加字母r或R表示原始字符串，表示字符串里面的\都不起作用了，即其中的特殊字符不进行转义，但字符串的最后一个字符不能是\。

# 回顾:原始字符串

```
>>> a = "C:\Users\xiaochunxiao\Desktop"
SyntaxError: (unicode error) 'unicodeescape' codec can't decode
bytes in position 2-3: truncated \UXXXXXXX escape
```

```
>>> a = 'C:\\Users\\xiaochunxiao\\Desktop'
>>> |
```

- 字符串界定符前面加字母r或R表示原始字符串，表示字符串里面的\都不起作用了，即其中的特殊字符不进行转义，但字符串的最后一个字符不能是\。

```
>>> a=r"C:\Users\xiaochunxiao\Desktop"
>>> a
'C:\\Users\\xiaochunxiao\\Desktop'
>>> print(a)
C:\Users\xiaochunxiao\Desktop
```

```
>>> a=r"C:\Users\xiaochunxiao\Desktop\"
SyntaxError: EOL while scanning string literal
```

交互式console在计算表达式时：如果表达式obj的值不为None时调用`print(repr(obj))`。对于字符串而言，输出结果为转义的字符串

`print(obj)`: 用户友好的输出，对于字符串而言，一串字符

- `repr(obj)`: 内部表示字符串

```
a=r"C:\Users\xiaochunxiao\Desktop"
print(a)
print(repr(a))
```

```
>>> a=r"C:\Users\xiaochunxiao\Desktop"
>>> a
'C:\\Users\\xiaochunxiao\\Desktop'
>>> print(repr(a))
'C:\\Users\\xiaochunxiao\\Desktop'
```

英文字母的ASCII码是连续的  
包括小写英文字母a..z、大写英文字母，十进制数字

# 回顾:字符串相关内置函数

转换为字符串  
转换为数字类型

| 函数         | 含义                                        | 实例                                                            | 结果                          |
|------------|-------------------------------------------|---------------------------------------------------------------|-----------------------------|
| bin(x)     | 把数字x转换为二进制字符串                             | bin(4)                                                        | '0b100'                     |
| oct(x)     | 把数字x转换为八进制字符串                             | oct(25)                                                       | '0o31'                      |
| hex(x)     | 把数字x转换为十六进制字符串                            | hex(75)                                                       | '0x4b'                      |
| str(obj)   | 把对象obj转换为字符串                              | str(16), str(1/2)                                             | ('16', '0.5')               |
| int(x[,d]) | 把数字x截取为整数，或将基数为d的数字型字符串x转换为整数，如果d没有缺省为十进制 | int(3.14), int('314')<br>int('ff',16)                         | (3,314)<br>255              |
| float(x)   | 将对象(数字或字符串)x转换为浮点数                        | float(3),float('3.14')<br>float('12e-2')                      | (3.0,3.14)<br>0.12          |
| len(obj)   | 返回序列类对象obj包含的元素个数                         | len('Hello World'),len('')                                    | (11, 0)                     |
| ord(s)     | 返回长度为1的字符串s中的字符对应的Unicode码（ASCII码是其子集）    | ord('a'),ord('A'),ord('你')<br>ord('0'), ord('\n')             | (97, 65, 20320)<br>(48, 10) |
| chr(x)     | 返回Unicode编码为x的字符串(len=1)                  | chr(65),chr(10),chr(20320)<br>chr(ord('d')-ord('a')+ord('A')) | ('A', '\n', '你')<br>'D'     |

# 回顾:其他方法

- 判断一个变量s是否为字符串：使用内置函数isinstance(s,str)
- 字符串属于不可变序列类型（通用序列方法）
  - 支持序列通用方法，包括下标和切片访问、成员关系、比较运算以及求字符串长度、最大值、最小值等
  - 作为iterable对象，当然也支持sorted/reversed/ enumerate/zip等内置函数
- 字符串属于不可变序列，不能直接对字符串对象进行元素增加、修改与删除等操作，切片操作也只能访问其中的元素而无法使用切片来修改字符串中的字符。

# 回顾:字符串支持的运算符 + \* %

## + 字符串合并成新的字符串

```
>>> a='abc' + '123'      #生成新对象 'abc123'
```

```
>>> b='\141b\x61' + '123'  # 'aba123', ord('\a')=97, 八进制为141,
    十六进制为61
```

\* 字符串和整数相乘相当于字符串的内容重复多次, 整数小于等于0时生成空字符串

```
>>> a='*- ' * 30      # '*****-
*****-'
```

```
>>> a= 4*'NE'         # 'NENENENE'
```

- **format\_string % obj** 把对象obj按格式要求format\_string转换为字符串。

# 索引和切片

|        |                               |                         |
|--------|-------------------------------|-------------------------|
| []     | 通过索引获取字符串中字符                  | >>>a[1]<br>'e'          |
| [:]    | 截取字符串中的一部分                    | >>>a[1:4]<br>'ell'      |
| in     | 成员运算符 - 如果字符串中包含给定的字符返回 True  | >>>"H" in a<br>True     |
| not in | 成员运算符 - 如果字符串中不包含给定的字符返回 True | >>>"M" not in a<br>True |

# Shell实验：

```
a = "Hello"
```

```
b = "Python"
```

```
print ("a + b 输出结果：", a + b)
```

```
print ("a * 2 输出结果：", a * 2)
```

```
print ("a[1] 输出结果：", a[1])
```

```
print ("a[1:4] 输出结果：", a[1:4])
```

```
print ("H" in a)
```

```
print ("M" not in a)
```

# 字符串格式化 %

```
print "My name is %s and weight is %d kg!" % ('Li Lei', 50)
```

```
>>> print "My name is %s and weight is %d kg!" % ('Li Lei', 50)
My name is Li Lei and weight is 50 kg!
```

`format_string % (value1, value2, ...)`

把 (value1, value2, ...) 中各对象按**格式化字符串format\_string**的要求转换为字符串

有几个%? 占位符，后面就跟几个变量或者值，顺序要对应好。如果只有一个%?，括号可以省略

# 字符串格式化

注意：除了%s对应的值是任意类型外，其他格式字符都要求值为相应的类型

- 常用格式字符——与待格式化的表达式类型匹配

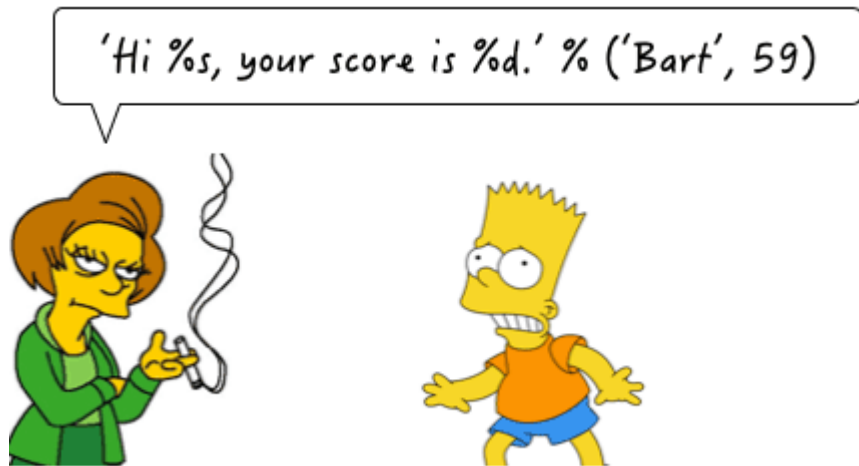
| 格式字符          | 说明               | 格式字符  | 说明                |
|---------------|------------------|-------|-------------------|
| %s            | 字符串(采用str()的显示)  | %x    | 十六进制整数            |
| %r            | 字符串(采用repr()的显示) | %e    | 指数(基底写为e)         |
| %c            | 单个字符             | %E    | 指数(基底写为E)         |
| <del>%b</del> | <del>二进制整数</del> | %f、%F | 浮点数               |
| %d            | 十进制整数            | %g    | 指数(e)或浮点数(根据显示长度) |
| %i            | 十进制整数            | %G    | 指数(E)或浮点数(根据显示长度) |
| %o            | 八进制整数            | %%    | 字符%               |

# Shell实验

- 输出不同类型的数据,观察错误提示
- 如果不确定应该用什么, %s永远起作用, 它会把任何数据类型转换为字符串

# 回顾:字符串的格式化

**format\_string % obj** 把对象obj按格式要求format\_string转换为字符串。如果格式化字符串中有多个占位符, 则obj应该是元组tuple对象, 格式为 (obj1, obj2..., objN)



```
>>> "Hi %s, your score is %d" % ('Bart', 59)
'Hi Bart, your score is 59'
>>>
```

| 格式字符                   | 含义                                                     | 示例                                                      | 结果                                |
|------------------------|--------------------------------------------------------|---------------------------------------------------------|-----------------------------------|
| %[width]s              | 输出字符串, width表示字符串长度, 不够前面填充空白                          | 'Hello %s' % 'Mike'                                     | 'Hello Mike'                      |
| %[width]d              | 输出整数字符串, width表示字符串长度, 不够前面填充空白                        | 'Length:%d' % 45                                        | 'Length:45'                       |
| %[width]c              | 输出字符 chr(num) width表示字符串长度, 不够前面填充空白                   | '%c' % 65                                               | 'A'                               |
| %[width][.precision]'f | 输出浮点数, 如果width和precision有表示字符串长度为width, 小数点后precision位 | '%f' % math.pi<br>'%.2f' % math.pi<br>'%7.4f' % math.pi | '3.141593'<br>'3.14'<br>' 3.1416' |

# 回顾：字符串格式化 %运算符

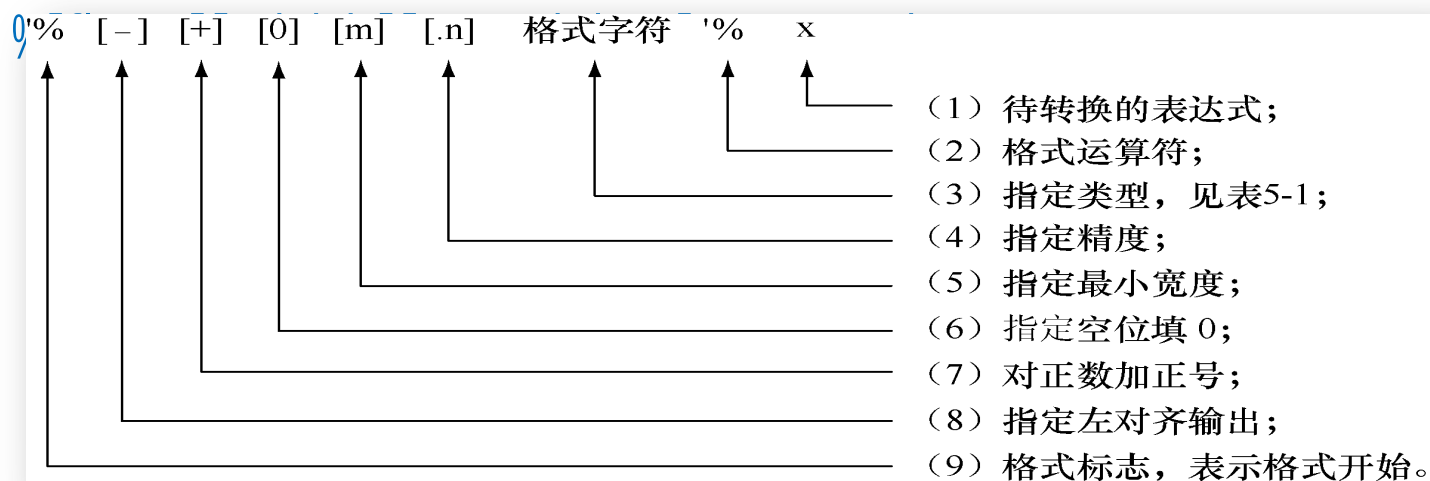
- 类似C语言的printf格式化输出：

format\_string % (value1, value2, ...)

把 (value1, value2, ...)中各对象按**格式化字符串format\_string**的要求转换为字符串

格式化字符串由固定文本以及格式说明符混合而成

格式说明符：



- flags: 对齐和填充等
  - [align-][sign+][#][0] 次序不规定
  - 默认右对齐。如果为-表示左对齐
  - #标志：type为xXo等时前面加上相应的进制表示前缀，对于浮点类型则要带小数点
- 宽度和精度可选，没有设置时采用系统缺省值

# 字符串格式化

注意：除了%s对应的值是任意类型外，其他格式字符都要求值为相应的类型

## • 常用格式字符——与待格式化的表达式类型匹配

| 格式字符          | 说明               | 格式字符  | 说明                |
|---------------|------------------|-------|-------------------|
| %s            | 字符串(采用str()的显示)  | %x    | 十六进制整数            |
| %r            | 字符串(采用repr()的显示) | %e    | 指数(基底写为e)         |
| %c            | 单个字符             | %E    | 指数(基底写为E)         |
| <del>%b</del> | <del>二进制整数</del> | %f、%F | 浮点数               |
| %d            | 十进制整数            | %g    | 指数(e)或浮点数(根据显示长度) |
| %i            | 十进制整数            | %G    | 指数(E)或浮点数(根据显示长度) |
| %o            | 八进制整数            | %%    | 字符%               |

```
>>> "%6.2s" % "python lesson"  
,  
    py  
>>> "%6.2s" % 65333  
,  
    65  
>>> |
```

```
>>> "%+10.2f" % 3.1415  
,  
    +3.14  
>>> "%+010.2f" % 3.1415  
,  
    +000003.14  
>>> |
```

# %常用格式举例

```
>>> "%6.2s" % "python lesson"  
,  
py  
>>> "%6.2s" % 65333  
,  
65  
>>> |
```

| 格式            | 说明                                          |
|---------------|---------------------------------------------|
| 10.2f         | 格式化浮点数，总宽度为至少10，四舍五入到小数点后第2位                |
| 10.2e         | 以科学计数法表示浮点数，总宽度至少为10，系数保留到小数点后2位            |
| 5d            | 将整数格式化为总宽度至少为5的十进制形式，不够填充空格                 |
| 05x           | 将整数格式化为总宽度至少为5的十六进制形式，不够填充0                 |
| 05o           | 将整数格式化为总宽度至少为5的八进制形式，不够填充0                  |
| <del>5b</del> | <del>将整数格式化为总宽度为5的二进制形式，</del> 新的format方法支持 |
| 50s           | 将字符串格式化为总宽度至少为50的字符串，不足则补上空格                |
| 50.2s         | 截取字符串的前2位，将其格式化为总宽度为50的字符串，不足补空格            |
| -10.2f        | 向左对齐格式化对象，默认右对齐                             |
| +10.2f        | 格式化浮点数，如果是正数，则自动加上正号+                       |

```
>>> "%+10.2f" % 3.1415  
,  
+3.14  
>>> "%+010.2f" % 3.1415  
,  
+000003.14  
>>> |
```

# 练习：字符串格式化例子

```
>>>a = 3.6674
```

```
>>>'%7.3f' % a  
' 3.667'
```

```
>>> "%d:%c"%(65,65)  
'65:A'
```

```
>>> 'My name is %s, and my age is %d'%( 'Li Lei',30)  
'My name is Li Lei , and my age is 30'
```

# 练习：字符串格式化例子

- `>>> pi = 3.141592653`
- `>>> print('%10.3f' % pi)` #字段宽10, 精度3
- `>>> print("pi = %.*f" % (3, pi))` #用\*从后面的元组中读取字段宽度或精度
- `>>> print('%010.3f' % pi)` #用0填充空白
- `>>> print('%-10.3f' % pi)` #左对齐
- `>>> print('%+f' % pi)` #显示正负号

# 字符串的format方法

# 字符串的format方法

- Python2支持, 也可用于Python3 : `format_string % (value1, value2, ...)`
- Python3引入 : `format_string.format(*args, **kwargs)`, 格式说明符有所变化 :
- `% [flags] [width] [.precision] type` 变为 `{[fieldname]![conversion][:formatspec]}`
  - `fieldname`: 确定 (自动或者人为指定) 当前位置格式化的值是对应的哪个位置参数或关键字参数, 参数后面还可以添加`.attr`或者`[index]`来访问参数的属性或者参数中下标对应的元素
  - `conversion`可取值`r`, `s`, `a`, 表示调用内置函数`repr`, `str`, `ascii`函数之后再格式化
  - `formatspec`: `[[fill]align][sign][#][0][width][,][.precision][typecode]` (次序规定)

align: 对齐方式

- < 左对齐 大部分对象缺省为左对齐
- > 右对齐 数字类型缺省为右对齐
- ^ 中间对齐
- = 用于数字类型, 表示填充字符在符号之后

- `fill`: 对齐时使用的填充字符, 缺省为空格
- `typecode`类似,
  - 新加入 `b` 将整数格式化成二进制字符串
  - `,` 用于数字类型, 表示每千位加逗号

# 字符串的format方法：例子

```
print( "The number {0:,,} in hex is: {0:#x},  
the number {1} in oct is  
{1:#o}".format( 5555, 55 ) )
```

```
print( "The number {1:,,} in hex is: {1:#x},  
the number {0} in oct is  
{0:#o}".format( 5555, 55 ) )
```

```
print( "my name is {name}, my age is {age},  
and my QQ is {qq}".format(name = "Dong  
Fuguo",age = 37,qq = "306467355") )
```

```
position = (5,8,13)
```

```
print( "X:{0[0]};Y:{0[1]};Z:{0[2]}".format(  
position) )
```

The number 5,555 in hex is: 0x15b3,  
the number 55 in oct is 0o67

The number 55 in hex is: 0x37, the  
number 5555 in oct is 0o12663

my name is Dong Fuguo, my age is  
37, and my QQ is 306467355

X:5;Y:8;Z:13

# 字符串的format方法：例子

```
weather = [("Monday", "rain"), ("Tuesday", "sunny"),  
           ("Wednesday", "sunny"), ("Thursday", "rain"), ("Friday", "Cloudy")]
```

```
formatter = "Weather of '{0[0]}' is '{0[1]}'.format #方法对象
```

```
for item in weather:  
    print(formatter(item))
```

Weather of 'Monday' is 'rain'  
Weather of 'Tuesday' is 'sunny'  
Weather of 'Wednesday' is 'sunny'  
Weather of 'Thursday' is 'rain'  
Weather of 'Friday' is 'Cloudy'

```
for item in map(formatter, weather):  
    print(item)
```

# 字符串编码

# 字符串编码格式

```
>>> chr(65), chr(10), chr(20320)
('A', '\n', '你')
>>> ord('a'), ord('A'), ord('你')
(97, 65, 20320)
>>> |
```

- 字符编码(encoding)：字符集中的字符在计算机内部存储时如何表示，映射为1个字节或者多个字节（称为code unit）
- 字符集(Charset):系统支持的所有抽象字符的集合，每个字符在该字符集中可以通过一个称为码点(Code Point)的数字唯一标识
- 最早的字符串编码是美国标准信息交换码**ASCII**（American Standard Code for Information Interchange），ASCII码采用**1个字节**来对字符进行编码，只用7位，最多只能表示128个符号。
  - 其他名字：cp367, US-ASCII
  - 128个字符组成，包括大小写字母、数字0-9、标点符号、非打印字符（换行符、制表符等4个）以及控制字符（退格、响铃等）组成
  - 编码成1个字节，最高位为0

键盘上每个字符在计算机内部都存储为一个整数，这个整数值就是字符的**ASCII码**：

| ASCII表                                                            |   |           |    |      |     |      |      |      |    |      |     |               |        |           |      |     |      |      |            |      |    |      |    |      |                            |
|-------------------------------------------------------------------|---|-----------|----|------|-----|------|------|------|----|------|-----|---------------|--------|-----------|------|-----|------|------|------------|------|----|------|----|------|----------------------------|
| ( American Standard Code for Information Interchange 美国标准信息交换代码 ) |   |           |    |      |     |      |      |      |    |      |     |               |        |           |      |     |      |      |            |      |    |      |    |      |                            |
| 高四位<br><br><br>低四位                                                |   | ASCII控制字符 |    |      |     |      |      |      |    |      |     |               |        | ASCII打印字符 |      |     |      |      |            |      |    |      |    |      |                            |
|                                                                   |   | 0000      |    |      |     |      |      | 0001 |    |      |     |               |        | 0010      | 0011 |     | 0100 | 0101 |            | 0100 |    | 0111 |    |      |                            |
|                                                                   |   | 0         |    |      |     |      |      | 1    |    |      |     |               |        | 2         | 3    |     | 4    | 5    |            | 6    |    | 7    |    |      |                            |
|                                                                   |   | 十进制       | 字符 | Ctrl | 代码  | 转义字符 | 字符解释 | 十进制  | 字符 | Ctrl | 代码  | 转义字符          | 字符解释   | 十进制       | 字符   | 十进制 | 字符   | 十进制  | 字符         | 十进制  | 字符 | 十进制  | 字符 | Ctrl |                            |
| 0000                                                              | 0 | 0         |    | ^@   | NUL | \0   | 空字符  | 16   | ▶  | ^P   | DLE |               | 数据链路转义 | 32        |      | 48  | 0    | 64   | @          | 80   | P  | 96   | `  | 112  | p                          |
| 0001                                                              | 1 | 1         | ☺  | ^A   | SOH |      | 标题开始 | 17   | ◀  | ^Q   | DC1 |               | 设备控制 1 | 33        | !    | 49  | 1    | 65   | A          | 81   | Q  | 97   | a  | 113  | q                          |
| 0010                                                              | 2 | 2         | ☹  | ^B   | STX |      | 正文开始 | 18   | ↕  | ^R   | DC2 |               | 设备控制 2 | 34        | "    | 50  | 2    | 66   | B          | 82   | R  | 98   | b  | 114  | r                          |
| 0011                                                              | 3 | 3         | ♥  | ^C   | ETX |      | 正文结束 | 19   | !! | ^S   | DC3 |               | 设备控制 3 | 35        | #    | 51  | 3    | 67   | C          | 83   | S  | 99   | c  | 115  | s                          |
| 0100                                                              | 4 | 4         | ♦  | ^D   | EOF |      | 传输结束 | 20   | ¶  | ^T   | DC4 |               | 设备控制 4 | 36        | \$   | 52  | 4    | 68   | D          | 84   | T  | 100  | d  | 116  | t                          |
| 0101                                                              | 5 | 5         | ♣  | ^E   | ENQ |      | 查询   | 21   | §  | ^U   | NAK |               | 否定应答   | 37        | %    | 53  | 5    | 69   | E          | 85   | U  | 101  | e  | 117  | u                          |
| 0110                                                              | 6 | 6         | ♠  | ^F   | ACK |      | 肯定应答 | 22   | —  | ^V   | SYN |               | 同步空闲   | 38        | &    | 54  | 6    | 70   | F          | 86   | V  | 102  | f  | 118  | v                          |
| 0111                                                              | 7 | 7         | •  | ^G   | BEL | \a   | 响铃   | 23   | ↕  | ^W   | ETB |               | 传输块结束  | 39        | '    | 55  | 7    | 71   | G          | 87   | W  | 103  | g  | 119  | w                          |
| 1000                                                              | 8 | 8         | ◼  | ^H   | BS  | \b   | 退格   | 24   | ↑  | ^X   | CAN |               | 取消     | 40        | (    | 56  | 8    | 72   | H          | 88   | X  | 104  | h  | 120  | x                          |
| 1001                                                              | 9 | 9         | ○  | ^I   | HT  | \t   | 横向制表 | 25   | ↓  | ^Y   | EM  |               | 介质结束   | 41        | )    | 57  | 9    | 73   | I          | 89   | Y  | 105  | i  | 121  | y                          |
| 1010                                                              | A | 10        | ◉  | ^J   | LF  | \n   | 换行   | 26   | →  | ^Z   | SUB |               | 替代     | 42        | *    | 58  | :    | 74   | J          | 90   | Z  | 106  | j  | 122  | z                          |
| 1011                                                              | B | 11        | ♂  | ^K   | VT  | \v   | 纵向制表 | 27   | ←  | ^[   | BSC | \e            | 溢出     | 43        | +    | 59  | ;    | 75   | K          | 91   | [  | 107  | k  | 123  | {                          |
| 1100                                                              | C | 12        | ♀  | ^L   | FF  | \f   | 换页   | 28   | └  | ^_   | FS  |               | 文件分隔符  | 44        | ,    | 60  | <    | 76   | L          | 92   | \  | 108  | l  | 124  |                            |
| 1101                                                              | D | 13        | ♪  | ^M   | CR  | \r   | 回车   | 29   | ↔  | ^]   | GS  |               | 组分隔符   | 45        | -    | 61  | =    | 77   | M          | 93   | ]  | 109  | m  | 125  | }                          |
| 1110                                                              | E | 14        | 🎵  | ^N   | SO  |      | 移出   | 30   | ▲  | ^^   | RS  |               | 记录分隔符  | 46        | .    | 62  | >    | 78   | N          | 94   | ^  | 110  | n  | 126  | ~                          |
| 1111                                                              | F | 15        | 🎵  | ^O   | SI  |      | 移入   | 31   | ▼  | ^-   | US  |               | 单元分隔符  | 47        | /    | 63  | ?    | 79   | O          | 95   | _  | 111  | o  | 127  | ␣<br>^Backspace<br>代码: DEL |
| 注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。                              |   |           |    |      |     |      |      |      |    |      |     |               |        |           |      |     |      |      |            |      |    |      |    |      |                            |
| 制作：NHL                                                            |   |           |    |      |     |      |      |      |    |      |     | QQ:1208980380 |        |           |      |     |      |      | 2013/08/08 |      |    |      |    |      |                            |

注：表中的ASCII字符可以用“Alt + 小键盘上的数字键”方法输入。

制作：NHL QQ:1208980380 2013/08/08

# 其他常见字符集

- ISO-8859-1 (Latin-1, cp1252): 西欧采用的 8 位字符集, 对于ASCII进行扩展, 最高位可以为 1, 供使用变音符号的拉丁字母语言使用
- GB2312: 中国国家标准简体中文字符集
  - 1个字节表示英文, 与ASCII一致
  - 2个字节表示中文, 其最高位都为1
- GBK对GB2312的扩充, 支持更多的汉字, 包括繁体汉字等, 而CP936是微软在GBK基础上开发的编码方式。GB2312、GBK和CP936都是使用2个字节表示中文
- GB18030: 加入少数民族字符

# 常见字符集：Unicode

```
>>> print('\u56fd')
国
>>> ord('国')
22269
>>> hex(22269)
'0x56fd'
>>> |
```

- Unicode(统一码): <http://www.unicode.org/>
  - 传统的编码方式给出了某个字符集中的字符与数字之间的映射关系，不同编码方式的映射不一样，而且所支持的字符也不一样
  - Unicode统一了全球不同语言采用的字符集，这些字符通过一个Unicode Code Point来描述
  - 内置函数ord返回字符对应的unicode，chr反之
  - 起初Unicode编码为16个比特，但从1996的Unicode 2.0之后编码范围扩展到U+0000..U+10FFFF，即21比特
  - Unicode字符可以采用UTF-8, UTF-16, UTF-32进行编码进行传输，将unicode对应的字符转换成字节序列
- UTF-8: Unicode transformation format
  - 目前使用最为广泛的UTF编码，Python3的字符串采用UTF-8编码.
  - 1~6个字节，ASCII字符采用1个字节，其ASCII编码与采用UTF-8编码的结果一致。中文采用3个字节

# Unicode 与utf-8转换

| Unicode符号范围<br>(十六进制) | UTF-8编码方式<br>(二进制)                  |
|-----------------------|-------------------------------------|
| -----+-----           |                                     |
| 0000 0000-0000 007F   | 0xxxxxxx                            |
| 0000 0080-0000 07FF   | 110xxxxx 10xxxxxx                   |
| 0000 0800-0000 FFFF   | 1110xxxx 10xxxxxx 10xxxxxx          |
| 0001 0000-0010 FFFF   | 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx |

解读 UTF-8 编码非常简单：如果一个字节的第一位是0，则这个字节单独就是一个字符；如果第一位是1，则连续有多少个1，就表示当前字符占用多少个字节。

- ‘严’ 的 Unicode 是4E25 (100111000100101)，根据上表，可以发现4E25处在第三行的范围内 (0000 0800 - 0000 FFFF) ，
- 因此 ‘严’ 的 UTF-8 编码需要三个字节，即格式是**1110xxxx 10xxxxxx 10xxxxxx**。
- 从‘严’ 的最后一个二进制位开始，依次从后向前填入格式中的x，多出的位补0。这样就得到了，严的 UTF-8 编码是**11100100 10111000 10100101**，转换成十六进制就是E4B8A5

# 系统缺省的编码方式

- sys和locale提供了相应的方法来查看

```
>>> import sys
```

```
>>> sys.getdefaultencoding() 缺省的字符串编码方式
```

```
'utf-8'
```

```
>>> import locale
```

```
>>> locale.getdefaultlocale() 获得本地化设置, (语言, 编码)
```

```
('zh_CN', 'cp936')
```

```
>>> locale.getpreferredencoding() 获得用户可能使用的字符集, cp936 = GBK
```

```
'cp936'
```

# 字符串str与字节串bytes之间的转换

- 类型bytes类似于str，也是一个不可变的有序序列类型，只是其元素为单个字节，而str中每个元素为单个字符
- unicode字符串→字节串：**`str_obj.encode(encoding='utf-8', errors='strict') -> bytes`**
  - 将unicode字符串str\_obj转换为相应编码方式的字节串，缺省编码为utf-8，常用的编码还包括ascii、latin1、gbk等
  - errors指出在编码过程中出现错误时怎么办，可取的值包括strict,ignore,replace,backslashreplace和xmlcharrefreplace等。其中strict表示抛出异常，ignore表示忽略并跳过无法编码的字符，replace表示无法编码时替换为缺省替换字符。
- 字节串→unicode字符串
  - **`bytes_obj.decode(encoding='utf-8', errors='strict') -> str`**
  - `str(bytes_or_buffer[, encoding[, errors]]) -> str` 按照指定编码方式将字节串转换为unicode字符串，注意如果只有一个参数str(obj)，相当于调用对象obj的\_\_str\_\_方法

第一版教材P101 Python3 字符串对象  
不再有encode和decode方法说法有误

# 字符串str与字节串bytes之间的转换

- 可以指定具体的编码方式

一个汉字采用GBK编码时需要2个字节

```
>>> s1 = 'Python程序设计'
>>> bs1 = s1.encode()
>>> len(s1)
10
>>> bs1
b'Python\xe7\xa8\x8b\xe5\xba\x8f\xe8\xae\xe8\xae\xa1'
>>> type(bs1)
<class 'bytes'>
>>> len(bs1)
18
```

```
>>> bs1_gbk = s1.encode(encoding='gbk')
>>> bs1_gbk
b'Python\xb3\xcc\xd0\xf2\xc9\xe8\xbc\xc6'

>>> s1_gbk = bs1_gbk.decode()
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xb3 in position 6:
invalid start byte

>>> s1_gbk_1 = bs1_gbk.decode(encoding='gbk')
>>> s1_gbk_2 = str(bs1_gbk, encoding='gbk')
>>> s1_gbk_1 == s1_gbk_2 == s1
True
```